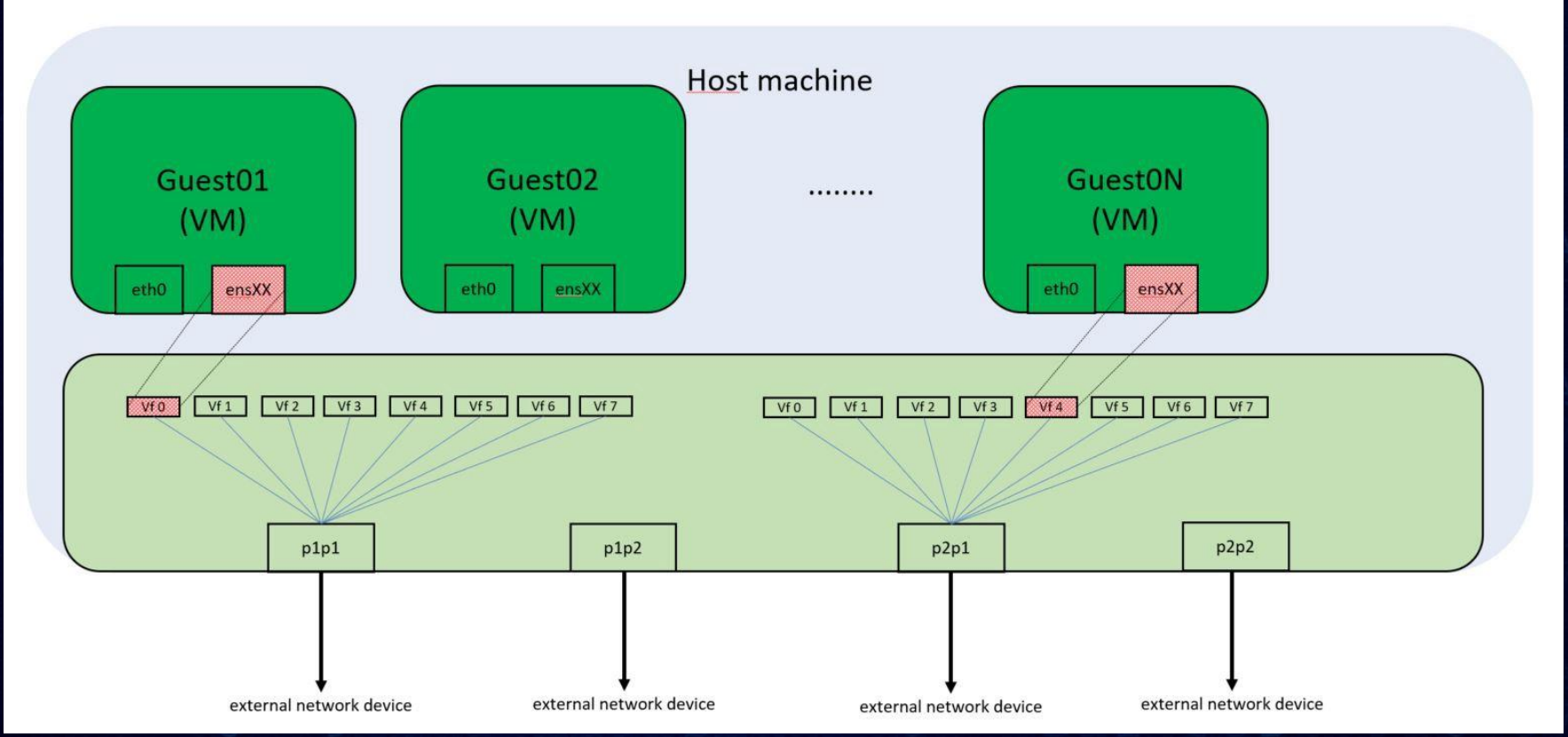
Security and Control of SR-IOV: What's Our Responsibility in the Kernel?

Carolyn Wyborny
Intel Corporation

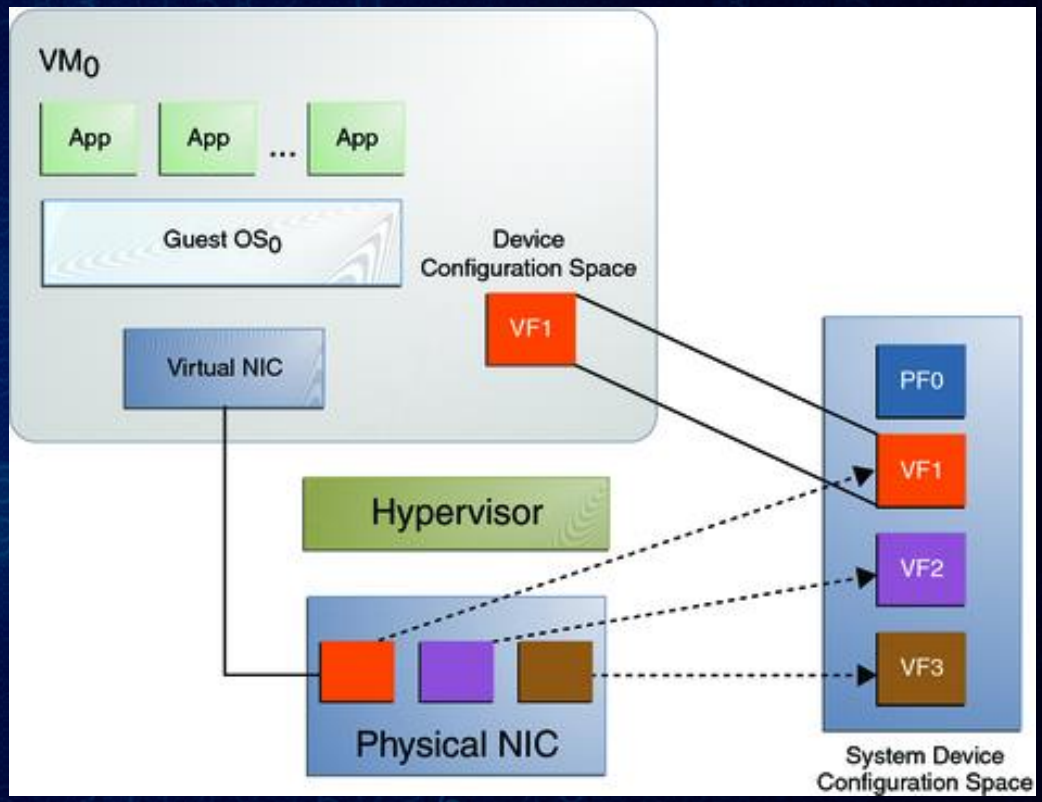
Background

SR-IOV provides for a dedicated, performant traffic path for multiple virtual functions (VF) in a networking device



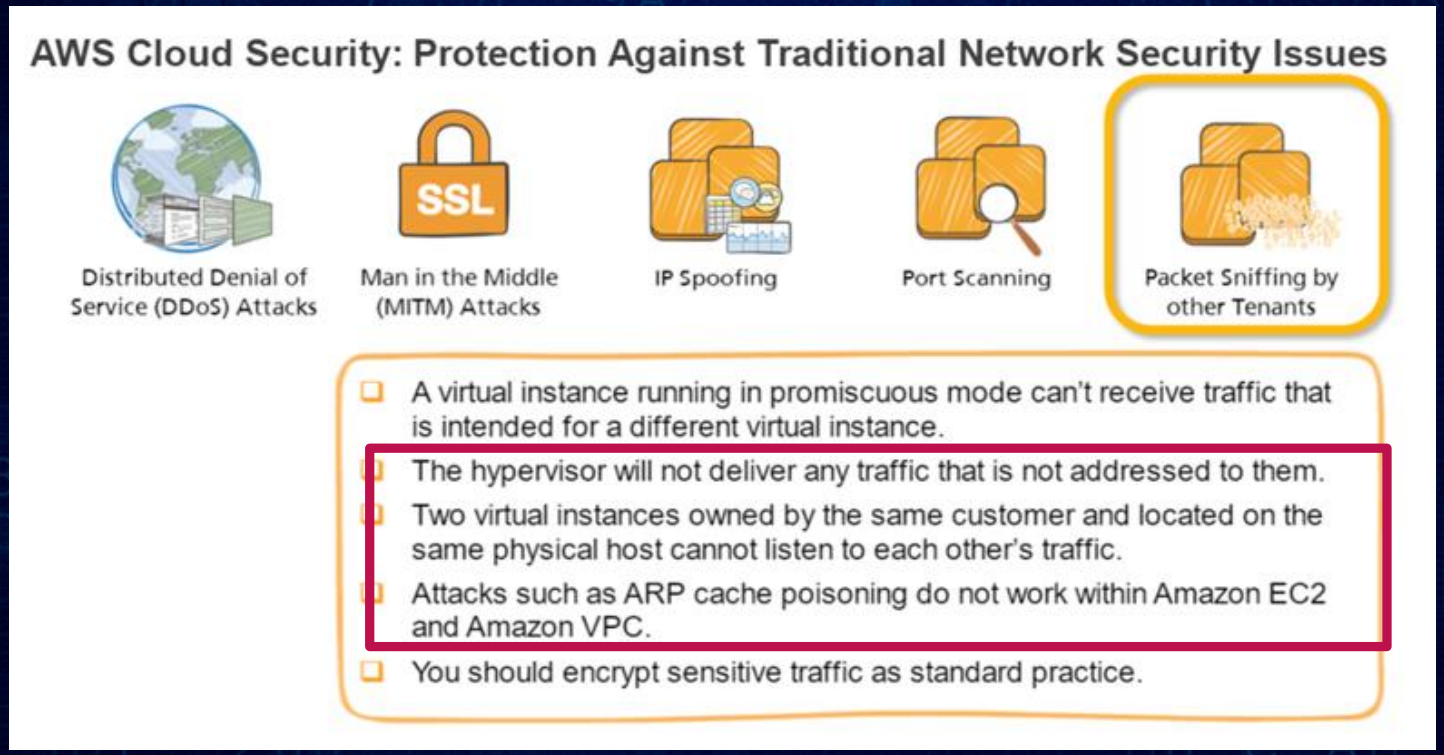
SR-IOV – Security and Port Isolation

- VF's have limited direct HW access
- Configuration managed by SW/FW
- Using SR-IOV provides more security than user space drivers



Virtualization Expectations

- VF isolation is expected by users and the SR-IOV specification



Comms Data Centers

- Virtualization in the telecom datacenter generates new and unique use cases
- Packet processing security model can differ from traditional eCommerce
- Tenant access to the networking HW or lack of it complicates the support picture



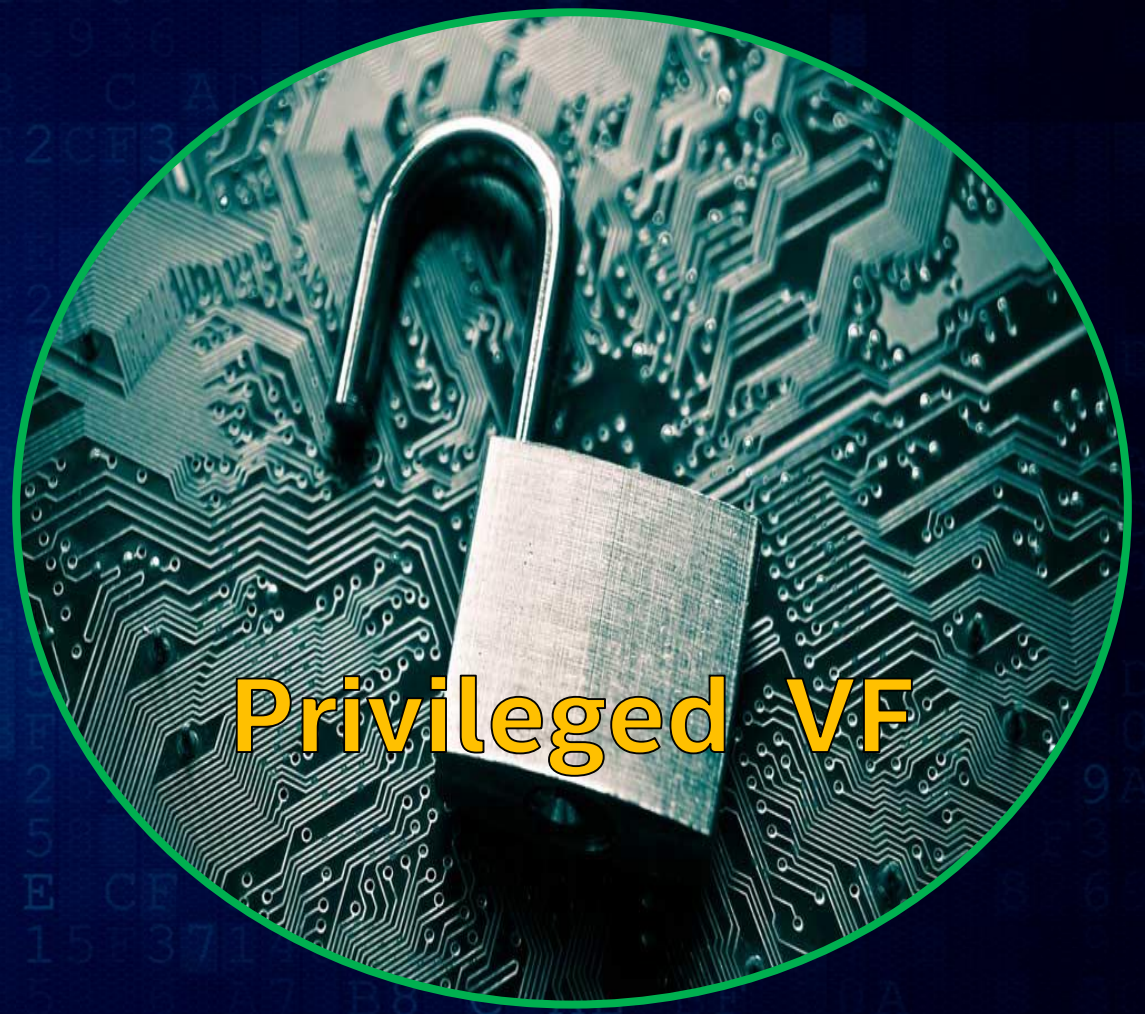
Today: Untrusted VF

- Traditional CSP model
- VM kernel is still in user space
- Datacenter doesn't control or access
- Malicious VF, etc.
- Safest model



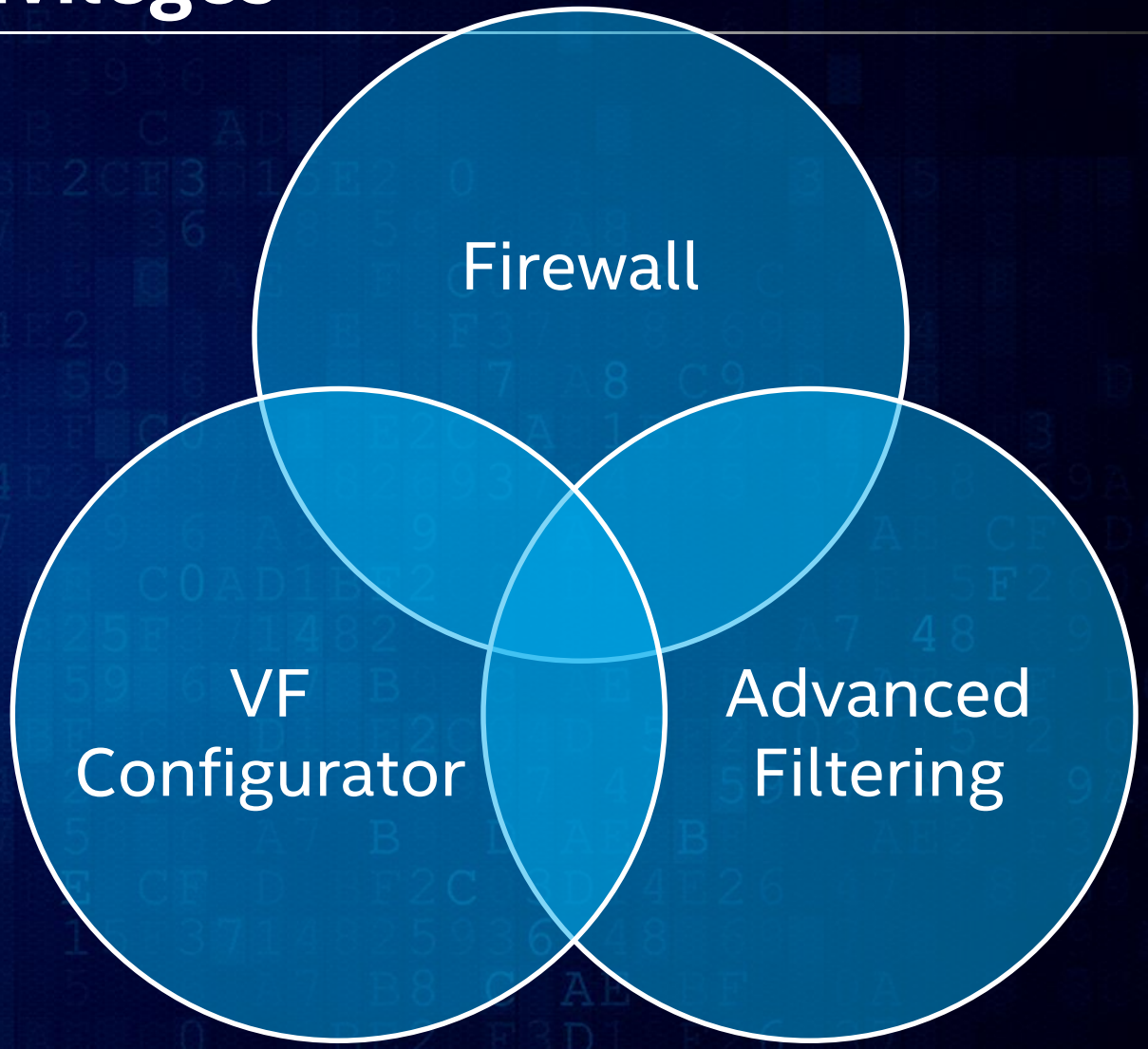
Privileged VF:

- Vendors define sensitive features
- Admins give VF more privileges with ip link set vf trust
- All or Nothing
- Users should define the policy, not device vendors



VF: Privileges and more Privileges

- Advanced traffic filtering
- Promiscuous mode and mirroring
- ACL/TCAM configuration
- Multiple VF configuration by a VF
- This is handled potentially differently by all the different device vendors.



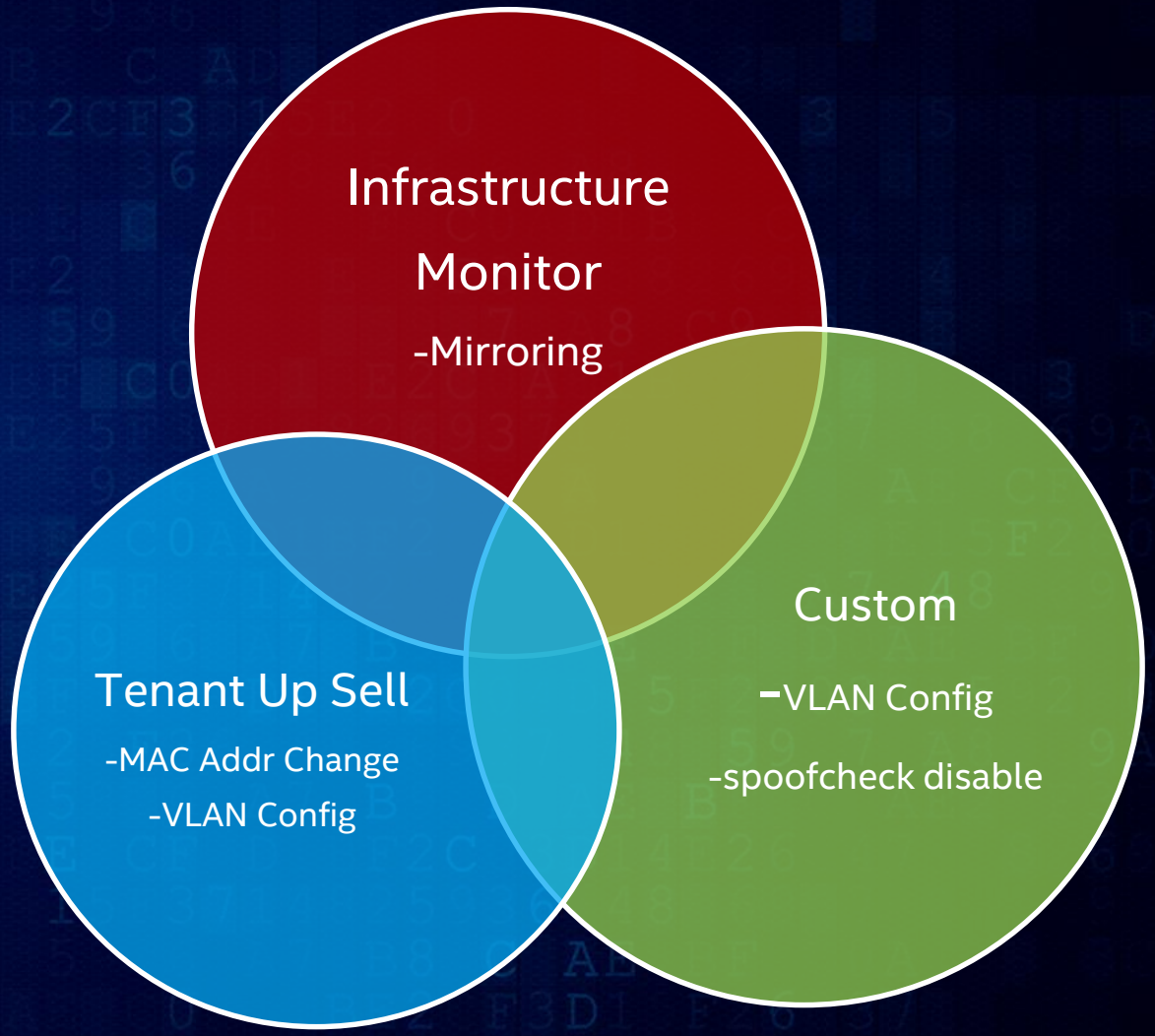
Additional Use Cases

VF features requested by users

- MAC Address changes
- VLAN configuration
- Spoofcheck disable

Generic defines could unify the user expectations

- Users can combine privileges for custom use cases
- Managed in the kernel in a consistent way



Proposal: Granular Trust using generic trust flags

- Trust flags are a bitfield of generic privileges for use per VF
- Extend VF get config callback with trust_flags
- Extension of the interface, no user space changes required
- Driver changes leave original “ALL OR NOTHING” policy in place.
- Makes VF trust model more flexible



VF Trust Flags – Generically defined

Base Set:

- VF_TRUST_ADV_FLOW – Filtering for traffic external to the VF
- VF_TRUST_MIRROR - Mirroring
- VF_TRUST_PROMISC - General promiscuous
- VF_TRUST_VLAN_PROM - VLAN promiscuous specifically
- VF_TRUST_MC_PROM – Multicast promiscuous specifically
- VF_TRUST_MACADDR_CHANGE - Allow MAC Address change
- VF_TRUST_MTU_CHANGE - Allow MTU changes
- VF_TRUST_MAC_SPFCHK_DIS - Allow MAC spoofcheck disable
- VF_TRUST_VLAN_SPFCHK_DIS - Allow VLAN spoofcheck disable

What else? RSS configuration?



Trust Flags Design

- Extend VF get config callback in kernel to allow trust flag configuration
- Add generic set of defines to the kernel for use in this configuration
- Add validation of generic flag configuration in the kernel interface
- Change iproute2 toolset



More Detailed Design

Kernel:

- Add ifla_trust_flags struct and flag definitions
- Extend ifla_vf_info to provide trust flags capability

Drivers:

- Change .ndo_set_vf_trust bool parameter to u32
- To enable trust flags, configure ifla_vf_info->trust_flags instead of ifla_vf_info->trusted in .ndo_get_vf_config
- Not changing driver's response in .ndo_get_vf_config keeps feature as currently designed.

ip link:

- Change user instructions and user interface to allow hex value in addition to "on" or "off"

Follow up:

- Patches submitted RFC
- Generic list of flags discussion with device vendors

